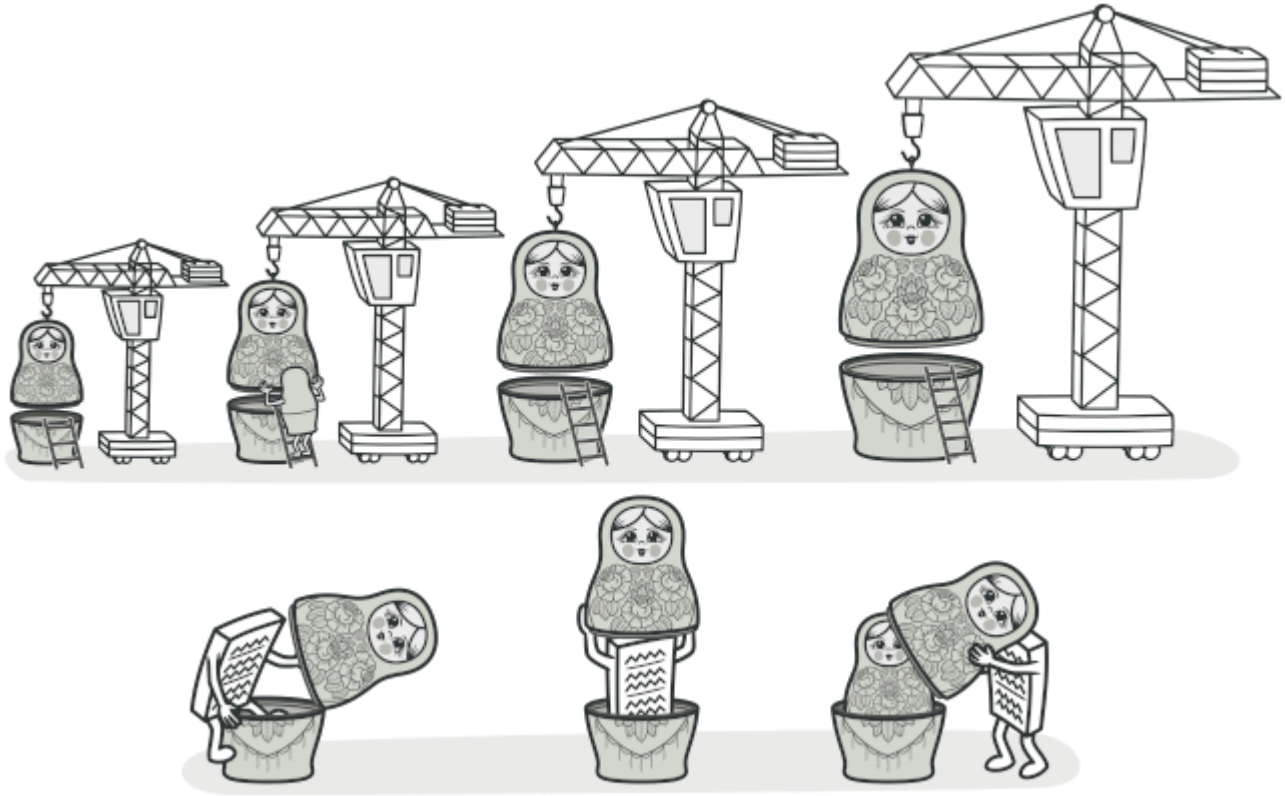
Liquidação de Outono

Decorador

Também conhecido como: Invólucro

Intenção

Decorator é um padrão de projeto estrutural que permite associar novos comportamentos a objetos, colocando esses objetos dentro de objetos encapsuladores especiais que contêm os comportamentos.



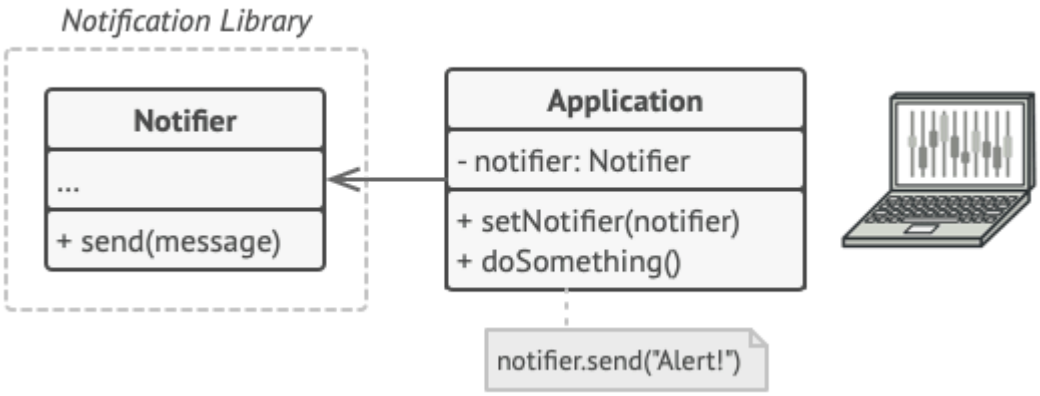
Problema

Imagine que você está trabalhando em uma biblioteca de notificações que permite que outros programas notifiquem seus usuários sobre eventos importantes.

A versão inicial da biblioteca era baseada em uma `Notifier` classe que possuía apenas alguns campos, um construtor e um único `send` método. O método podia receber um argumento de mensagem de um cliente e

Liquidação de Outono

construtor. Um aplicativo de terceiros, que atuava como cliente, deveria criar e configurar o objeto notificador uma única vez e, em seguida, utilizá-lo sempre que algo importante acontecesse.



Um programa poderia usar a classe de notificação para enviar notificações sobre eventos importantes para um conjunto predefinido de endereços de e-mail.

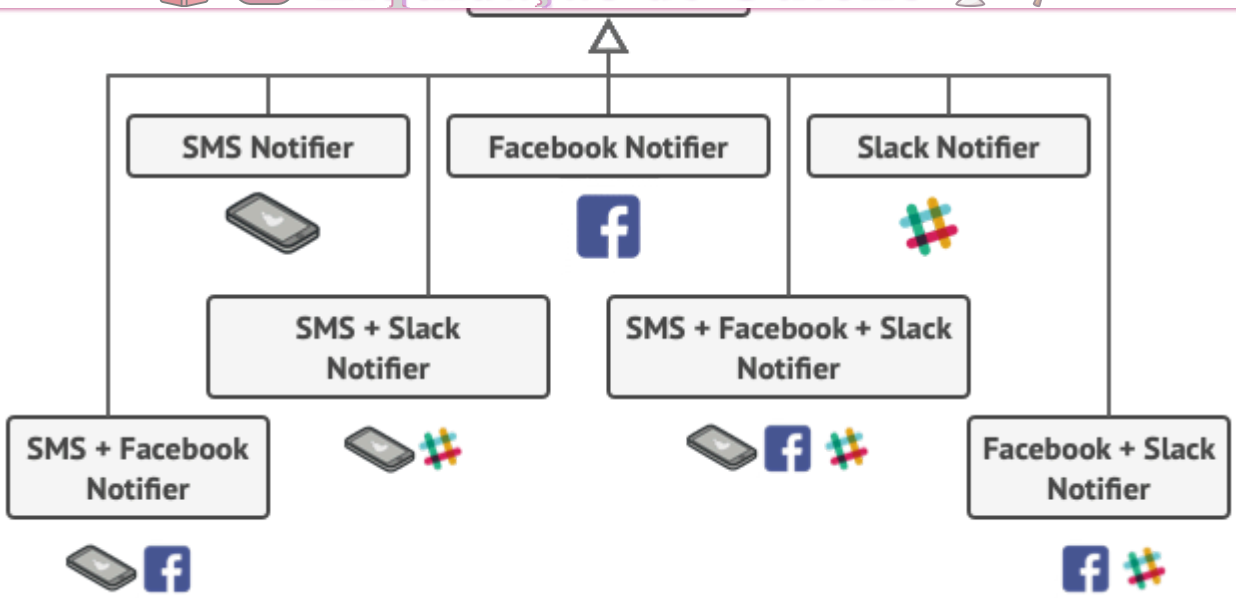
Em algum momento, você percebe que os usuários da biblioteca esperam mais do que apenas notificações por e-mail. Muitos gostariam de receber um SMS sobre assuntos críticos. Outros gostariam de ser notificados pelo Facebook e, claro, os usuários corporativos adorariam receber notificações pelo Slack.

Cada tipo de notificação é implementado como uma subclasse de um notificador.

Qual a dificuldade? Você estendeu a `Notifier` classe e colocou os métodos de notificação adicionais em novas subclasses. Agora, o cliente deveria instanciar a classe de notificação desejada e usá-la para todas as notificações subsequentes.

Mas aí alguém, com razão, perguntou: "Por que não usar vários tipos de notificação ao mesmo tempo? Se sua casa estivesse pegando fogo, você provavelmente gostaria de ser informado por todos os canais."

Você tentou resolver esse problema criando subclasses especiais que combinavam vários métodos de notificação em uma única classe. No entanto, logo ficou evidente que essa abordagem inflaria o código imensamente, não apenas o código da biblioteca, mas também o código do cliente.



Explosão combinatória de subclasses.

Você precisa encontrar outra maneira de estruturar as classes de notificações para que seu número não quebre acidentalmente algum recorde do Guinness.

😊Solução

Estender uma classe é a primeira coisa que vem à mente quando você precisa alterar o comportamento de um objeto. No entanto, a herança tem várias ressalvas importantes que você precisa conhecer.

- A herança é estática. Você não pode alterar o comportamento de um objeto existente em tempo de execução. Você só pode substituir o objeto inteiro por outro criado a partir de uma subclasse diferente.
- Subclasses podem ter apenas uma classe pai. Na maioria das linguagens, a herança não permite que uma classe herde comportamentos de múltiplas classes simultaneamente.

Uma das maneiras de superar essas limitações é usando *agregação* ou *composição.* ⓘEm vez de *herança* . Ambas as alternativas funcionam de maneira quase idêntica: um objeto *tem uma* referência a outro e delega a ele algumas tarefas, enquanto que, com a herança, o próprio objeto *é* capaz de realizar essa tarefa, herdando o comportamento de sua superclasse.

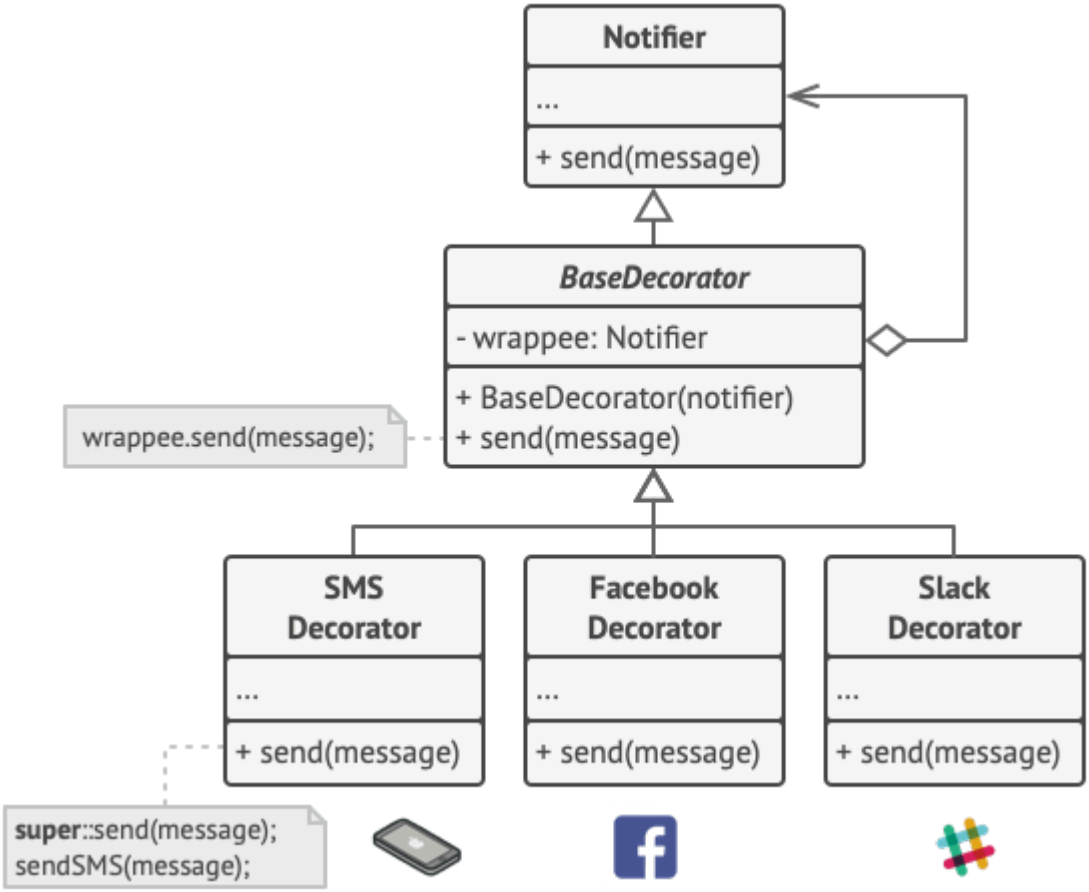
Com essa nova abordagem, você pode facilmente substituir o objeto "auxiliar" vinculado por outro, alterando o comportamento do contêiner em tempo de execução. Um objeto pode usar o comportamento de várias classes, tendo referências a múltiplos objetos e delegando a eles todos os tipos de trabalho. Agregação/composição é o princípio fundamental por trás de muitos padrões de projeto, incluindo o Decorator. Dito isso, vamos retornar à discussão sobre padrões.

Herança versus agregação

"Wrapper" é um apelido alternativo para o padrão Decorator que expressa claramente a ideia principal do padrão. Um *wrapper* é um objeto que pode ser vinculado a um objeto *alvo* . O wrapper contém o mesmo conjunto de métodos que o alvo e delega a ele todas as requisições que recebe. No entanto, o wrapper pode alterar o resultado executando alguma ação antes ou depois de passar a requisição para o alvo.

Quando um simples wrapper se torna um decorador de fato? Como mencionei, o wrapper implementa a mesma interface que o objeto encapsulado. É por isso que, da perspectiva do cliente, esses objetos são idênticos. Faça com que o campo de referência do wrapper aceite qualquer objeto que siga essa interface. Isso permitirá que você cubra um objeto com múltiplos wrappers, adicionando o comportamento combinado de todos eles.

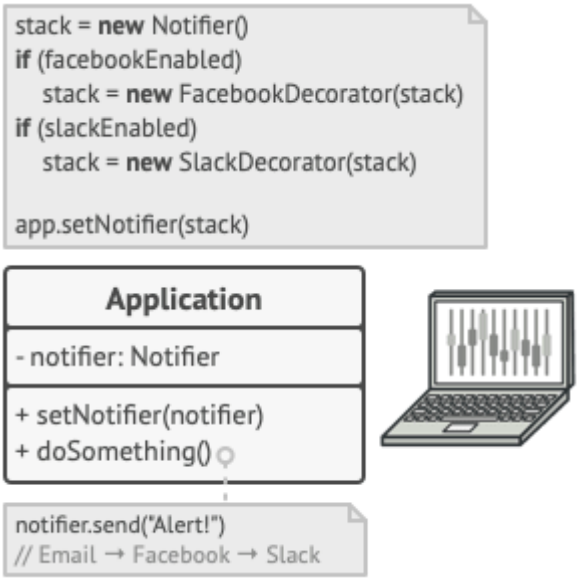
Em nosso exemplo de notificações, vamos manter o comportamento simples de notificação por e-mail dentro da `Notifier` classe base, mas transformar todos os outros métodos de notificação em decoradores.



Diversos métodos de notificação se tornam elementos decorativos.

Liquidação de Outono

que correspondam às preferências do cliente. Os objetos resultantes serão estruturados como uma pilha.

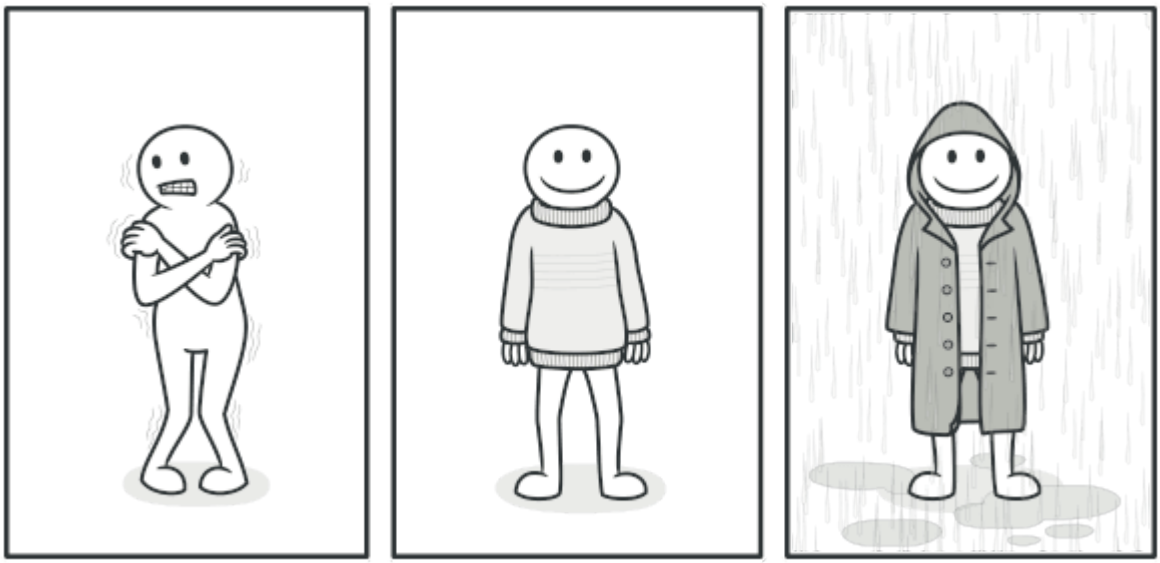


Os aplicativos podem configurar conjuntos complexos de decoradores de notificação.

O último decorador na pilha seria o objeto com o qual o cliente realmente interage. Como todos os decoradores implementam a mesma interface que o notificador base, o restante do código do cliente não se importará se está trabalhando com o objeto notificador "puro" ou com o decorado.

Poderíamos aplicar a mesma abordagem a outros comportamentos, como formatar mensagens ou compor a lista de destinatários. O cliente pode decorar o objeto com quaisquer decoradores personalizados, desde que sigam a mesma interface que os demais.

Analogia com o mundo real

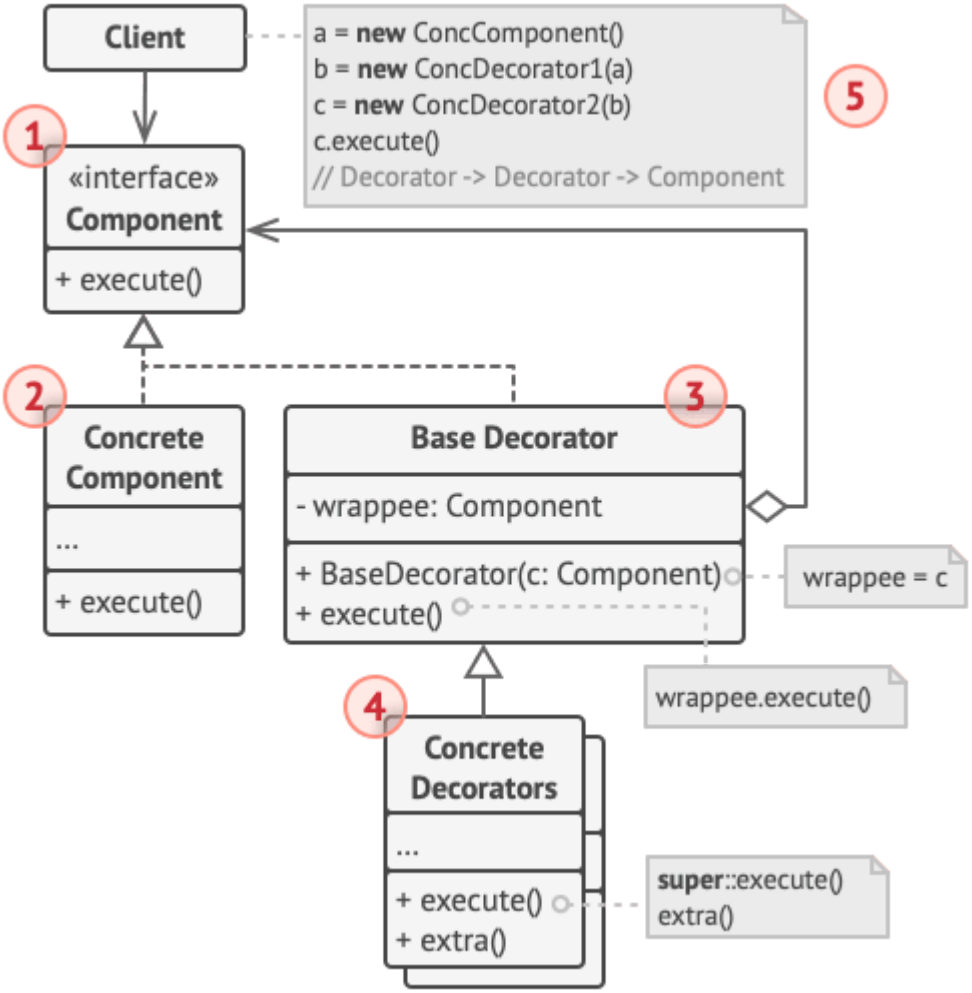


O uso de várias peças de roupa proporciona um efeito combinado.

Liquidação de Outono

suéter. Se ainda estiver com frio mesmo de suéter, pode vestir uma jaqueta por cima. Se estiver chovendo, pode colocar uma capa de chuva. Todas essas peças de roupa "ampliam" seu comportamento básico, mas não fazem parte de você, e você pode facilmente tirar qualquer peça de roupa sempre que não precisar dela.

Estrutura



1. O **componente** declara a interface comum tanto para os elementos encapsulados quanto para os elementos encapsulados.

2. **Um componente concreto** é uma classe de objetos que está sendo encapsulada. Ele define o comportamento básico, que pode ser alterado por decoradores.

3. A classe **Decorator base** possui um campo para referenciar um objeto encapsulado. O tipo do campo deve ser declarado como a interface do componente para que possa conter tanto componentes concretos quanto decoradores. O decorador base delega todas as operações ao objeto encapsulado.

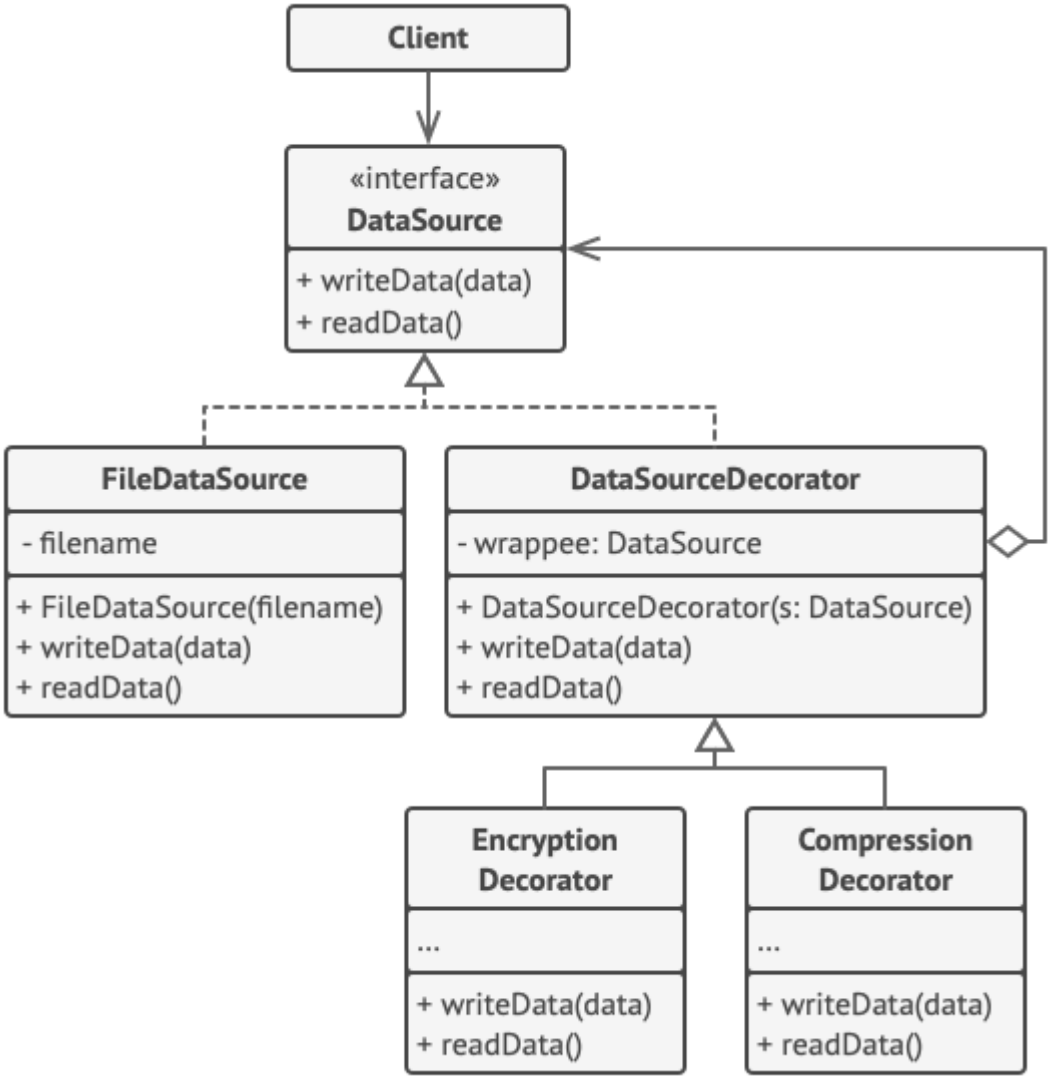
4. **Decoradores concretos** definem comportamentos extras que podem ser adicionados dinamicamente aos componentes. Decoradores concretos sobrescrevem os métodos do decorador base e executam seu comportamento antes ou depois da chamada do método pai.

Liquidação de Outono

todos os objetos através da interface do componente.

#Pseudocódigo

Neste exemplo, o padrão **Decorator** permite compactar e criptografar dados sensíveis independentemente do código que realmente utiliza esses dados.



Exemplo de decoradores de criptografia e compressão.

A aplicação envolve o objeto de fonte de dados com um par de decoradores. Ambos os decoradores alteram a forma como os dados são gravados e lidos do disco:

- Pouco antes dos dados serem **gravados em disco** , os decoradores os criptografam e compactam. A classe original grava os dados criptografados e protegidos no arquivo sem ter conhecimento da alteração.

- Logo após os dados serem **lidos do disco** , eles passam pelos mesmos decoradores, que os descompactam e decodificam.

Liquidação de Outono

intercambiáveis no código do cliente.

```
// A interface do componente define operações que podem ser
// alteradas por decoradores.
interface DataSource is
    method writeData ( data )
    method readData ( ) : data

// Componentes concretos fornecem implementações padrão para as
// operações. Pode haver várias variações dessas
// classes em um programa.
class FileDataSource implements DataSource is
    constructor FileDataSource ( filename ) { . . . } method writeData ( data ) is // Escreve da
```

```
// Chamada para um objeto encapsulado.
class EncryptionDecorator extends DataSourceDecorator is
    method writeData ( data ) is
        // 1. Criptografa os dados passados.
        // 2. Passa os dados criptografados para o
```

```
    method  readData ( ) : data  is
        // 1. Obtém os dados do método readData do objeto encapsulado.
        // 2. Tenta descriptografá-los, se estiverem criptografados.
        // 3. Retorna o resultado.

// Você pode encapsular objetos em várias camadas de decoradores.
class  CompressionDecorator  extends  DataSourceDecorator  is
    method  writeData ( data )  is
        // 1. Comprime os dados passados.
        // 2. Passa os dados comprimidos para o
        método writeData do objeto encapsulado. //

    method  readData ( ) : data  is
        // 1. Obtém os dados do método readData do objeto encapsulado.
        // 2. Tenta descomprimi-los, se estiverem comprimidos.
        // 3. Retorna o resultado.


// Opção 1. Um exemplo simples de um assembly de decoradores.
class  Application  is
    method  dumbUsageExample ( )  is
        source  =  new  FileDataSource ( " somefile.data " ) source.writeData ( salaryRecords ) // (

// Especificidades de armazenamento . Funcionam com uma fonte de dados pré-configurada // recebida (
class  SalaryManager  is
    field  source :  DataSource

    constructor  SalaryManager ( source :  DataSource )  {  . . . } method load ( ) is return sourc
```

Aplicabilidade

Utilize o padrão Decorator quando precisar atribuir comportamentos extras a objetos em tempo de execução sem quebrar o código que utiliza esses objetos.

O Decorator permite estruturar sua lógica de negócios em camadas, criar um decorator para cada camada e compor objetos com várias combinações dessa lógica em tempo de execução. O código do cliente pode tratar todos esses objetos da mesma maneira, já que todos seguem uma interface comum.

Utilize esse padrão quando for complicado ou impossível estender o comportamento de um objeto usando herança.

Muitas linguagens de programação possuem a `final` palavra-chave `new`, que pode ser usada para impedir a extensão de uma classe. Para uma classe final, a única maneira de reutilizar o comportamento existente seria envolver a classe com um wrapper próprio, utilizando o padrão Decorator.

Como implementar

1. Certifique-se de que seu domínio de negócios possa ser representado como um componente primário com várias camadas opcionais sobrepostas.

2. Identifique os métodos comuns ao componente principal e às camadas opcionais. Crie uma interface de componente e declare esses métodos nela.

3. Crie uma classe de componente concreta e defina o comportamento básico nela.

Liquidação de Outono

encapsulado. O campo deve ser declarado com o tipo de interface do componente para permitir a vinculação a componentes concretos, bem como a decoradores. O decorador base deve delegar todo o trabalho ao objeto encapsulado.

5. Certifique-se de que todas as classes implementem a interface do componente.

6. Crie decoradores concretos estendendo-os a partir do decorador base. Um decorador concreto deve executar seu comportamento antes ou depois da chamada ao método pai (que sempre delega ao objeto encapsulado).

7. O código do cliente deve ser responsável por criar os decoradores e compô-los da maneira que o cliente precisar.

⚖Prós e contras

- ✓ Você pode estender o comportamento de um objeto sem criar uma nova subclasse.
- ✓ Você pode adicionar ou remover responsabilidades de um objeto em tempo de execução.
- ✓ Você pode combinar vários comportamentos envolvendo um objeto em múltiplos decoradores.
- ✓ *Princípio da Responsabilidade Única* . Você pode dividir uma classe monolítica que implementa muitas variantes possíveis de comportamento em várias classes menores.

- ✗ É difícil remover um wrapper específico da pilha de wrappers.
- ✗ É difícil implementar um decorador de forma que seu comportamento não dependa da ordem na pilha de decoradores.
- ✗ O código de configuração inicial das camadas pode parecer bastante complexo.

⇄Relações com outros padrões

- **O padrão Adapter** oferece uma interface completamente diferente para acessar um objeto existente. Por outro lado, com o padrão **Decorator,** a interface permanece a mesma ou é estendida. Além disso, *o Decorator* suporta composição recursiva, o que não é possível quando se usa *o Adapter* .

- Com **o Adapter,** você acessa um objeto existente por meio de uma interface diferente. Com **o Proxy** , a interface permanece a mesma. Com **o Decorator,** você acessa o objeto por meio de uma interface aprimorada.

- **Os padrões Chain of Responsibility** e **Decorator** possuem estruturas de classe muito semelhantes. Ambos se baseiam na composição recursiva para passar a execução por uma série de objetos. No entanto, existem diversas diferenças cruciais.

Liquidação de Outono

também podem interromper o encaminhamento da requisição a qualquer momento. Por outro lado, vários *Decoradores* podem estender o comportamento do objeto, mantendo-o consistente com a interface base. Além disso, os Decoradores não podem interromper o fluxo da requisição.

- **Os tipos Composite** e **Decorator** possuem diagramas de estrutura semelhantes, uma vez que ambos dependem da composição recursiva para organizar um número ilimitado de objetos.

  Um *Decorator* é semelhante a um *Composite* , mas possui apenas um componente filho. Há outra diferença significativa: *o Decorator* adiciona responsabilidades extras ao objeto encapsulado, enquanto *o Composite* apenas "soma" os resultados de seus filhos.

  No entanto, os padrões também podem cooperar: você pode usar *o Decorator* para estender o comportamento de um objeto específico na árvore *Composite* .

- Projetos que fazem uso intensivo de **Composite** e **Decorator** podem se beneficiar do uso de **Prototype** . Aplicar esse padrão permite clonar estruturas complexas em vez de reconstruí-las do zero.

- **Decorator** permite que você altere a aparência de um objeto, enquanto **Strategy** permite que você altere suas entranhas.

- **Decorator** e **Proxy** possuem estruturas semelhantes, mas intenções muito diferentes. Ambos os padrões são construídos sobre o princípio da composição, onde um objeto deve delegar parte do trabalho a outro. A diferença é que um *Proxy* geralmente gerencia o ciclo de vida de seu objeto de serviço por conta própria, enquanto a composição de *Decorators* é sempre controlada pelo cliente.

</>Exemplos de código

Por que comprar o melhor e-book de padrões de design de todos os tempos?

- **Iniciantes:** Aprendam os padrões básicos e como eles funcionam!
- **Especialistas:** Use-o como uma referência prática e pesquisável!
- **Independente de linguagem:** exemplos em PHP, C#, Java e muito mais!

Liquidação de Outono

- 409 páginas bem estruturadas, de fácil leitura e sem jargões.
- 225 ilustrações e diagramas claros e úteis.

Saber mais...

© 2014-2025 Refactoring.Guru . Todos os direitos reservados.
Ilustrações de Dmitry Zhart

Escritório ucraniano: FOP Olga Skobeleva Abolmasova 7 Kiev, Ucrânia, 02002

E-mail: support@refactoring.guru

Escritório espanhol: Oleksandr Shvets Avda Pamplona 64 Pamplona, Espanha, 31009

E-mail: support@refactoring.guru